Sprawozdanie z projektu Evolutionary Computations

Autorzy

- Mateusz Bobula
- Norbert Dziwak
- Jakub Konopka

1. Technologie użyte w projekcie

- Python 3.12+
- PyQt6 do GUI
- benchmark-functions - funkcja testowa Hypersphere
- ruff - formater kodu
- uv - package manager

2. Wymagania środowiskowe

- Python 3.12
- venv z zainstalowanymi zależnościami z pyproject.toml
- uv

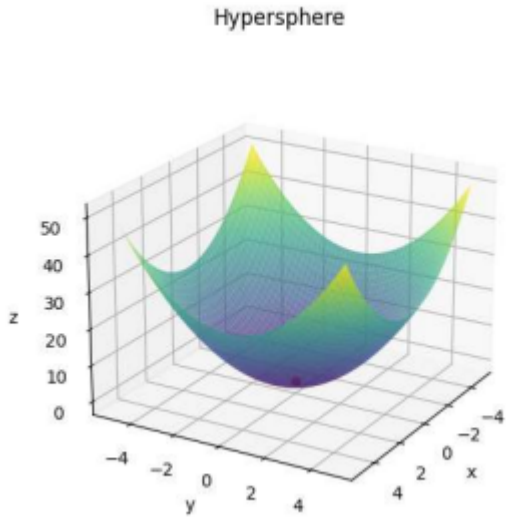
Uruchamianie aplikacji komendą uv run .\src\main.py

3. Opis wybranych funkcji testowych i ich optima

W aktualnej wersji aplikacji używana jest jedna funkcja testowa Hypersphere

Wzór funkcji:

$$f(x) = \sum_{i=1}^{N} x_i^2$$



- Funkcja jest separowalna i wypukła

- Wartość funkcji jest nieujemna
- Minimum globalne występuje w punkcie x = (0,...,0)
- W minimum globalnym wartość funkcji wynosi f(x) = 0

4. Wykresy wyników

Projekt ma gotowy mechanizm zapisu i wizualizacji przebiegu optymalizacji

- Dla każdego uruchomienia tworzony jest osobny katalog wyników
- Zapisywany jest plik summary.json z konfiguracją i wynikiem końcowym
- Zapisywany jest plik history.csv z najlepszym fitness dla każdej epoki
- Generowany jest plik fitness_history.svg jako wykres historii fitness

Osadzone wykresy

Wspólny config dla wszystkich wykresów

Na podstawie plików summary.json wszystkie sześć przebiegów ma ten sam config bazowy, zmiany co do bazowego configu widnieją zaraz pod nagłówkiem wykresu.

Parametr	Wartość
function	Hypersphere
a	-5.0
b	5.0
dimensions	3
precision	6
population_size	50
epochs	175
crossover_method	single_point
crossover_prob	0.8
mutation_method	single_point
mutation_prob	0.01
inversion_prob	0.05
elite_size	1
minimize	true

Wykres 1

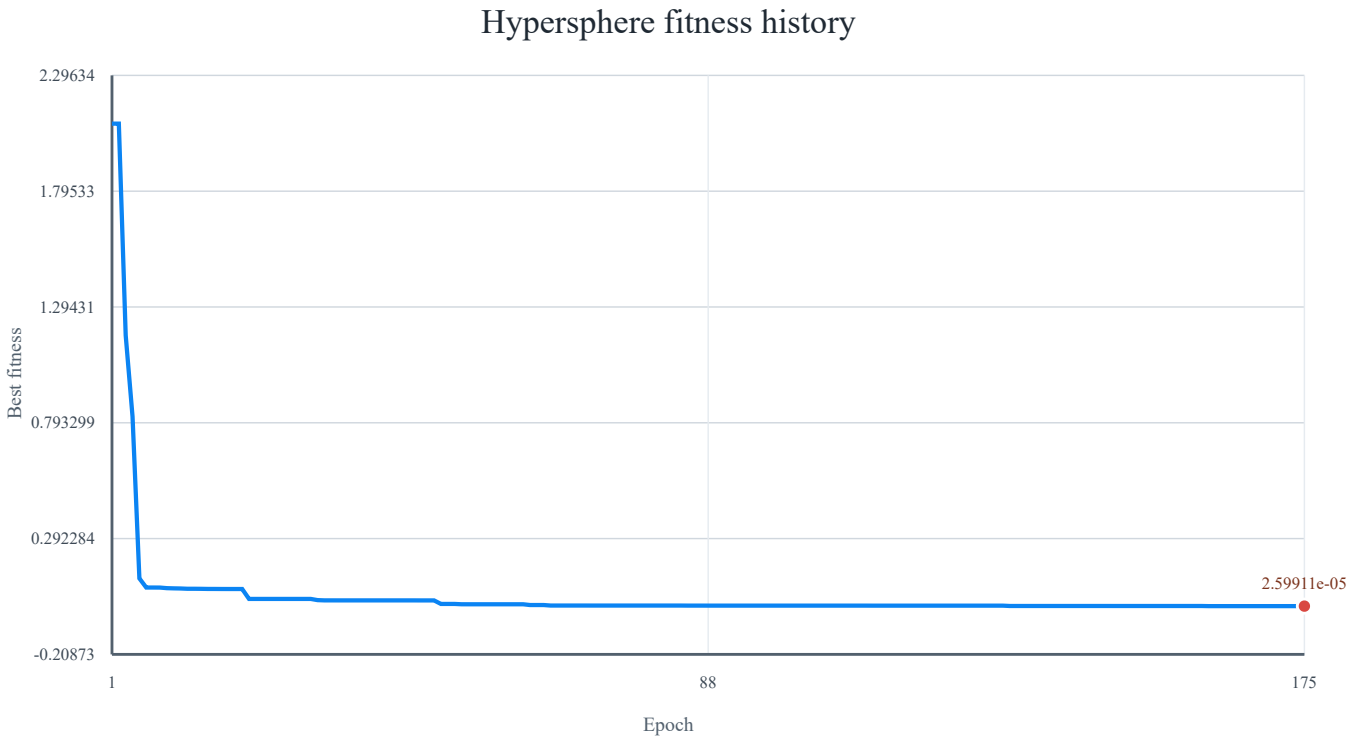
Wykres dla selekcji `tournament`

Kluczowa wartość configu względem pozostałych

- selection_method = tournament

Co widać na wykresie

- Najmocniejsza poprawa jakości w pierwszych kilku epokach
- Potem dłuższa faza stabilizacji



Wykres 2

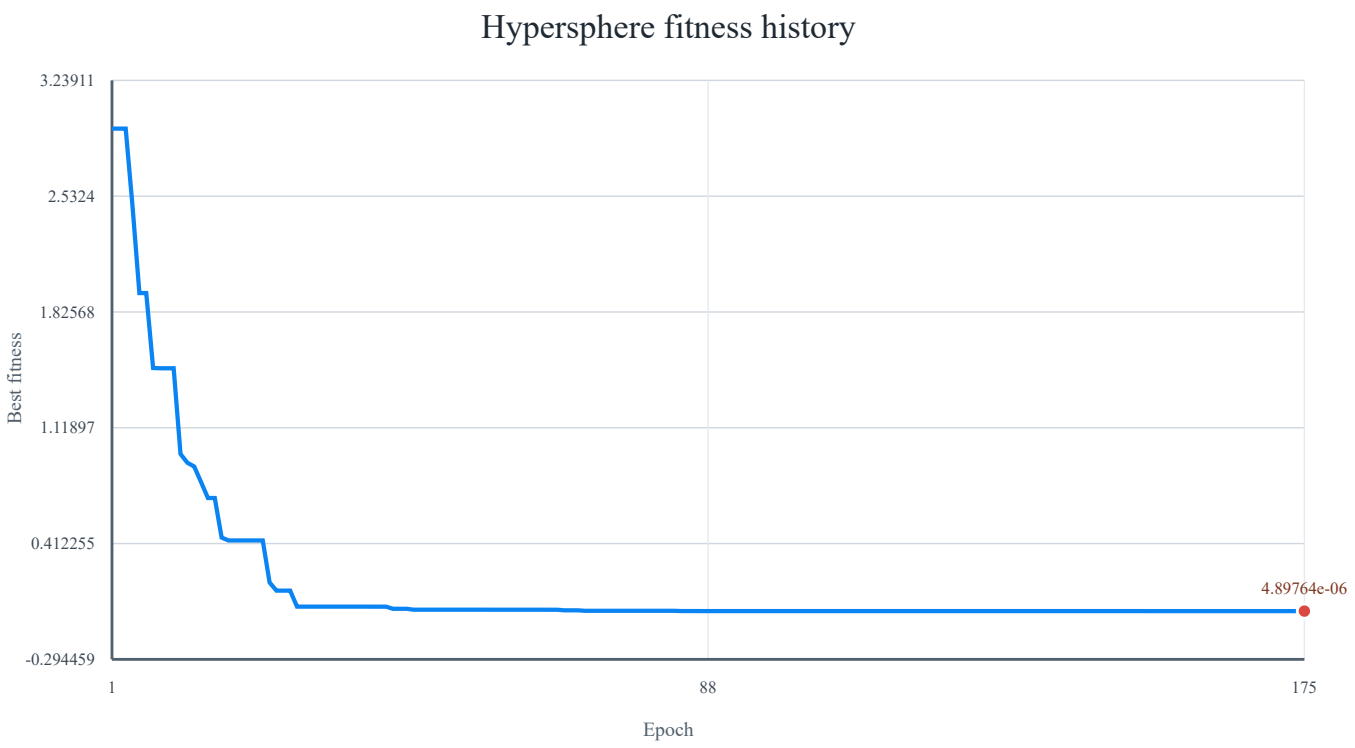
Wykres dla selekcji `best`

Kluczowa wartość configu względem pozostałych

- selection_method = best

Co widać na wykresie

- Szybkie dojście schodkowe do wyniku



Wykres 3

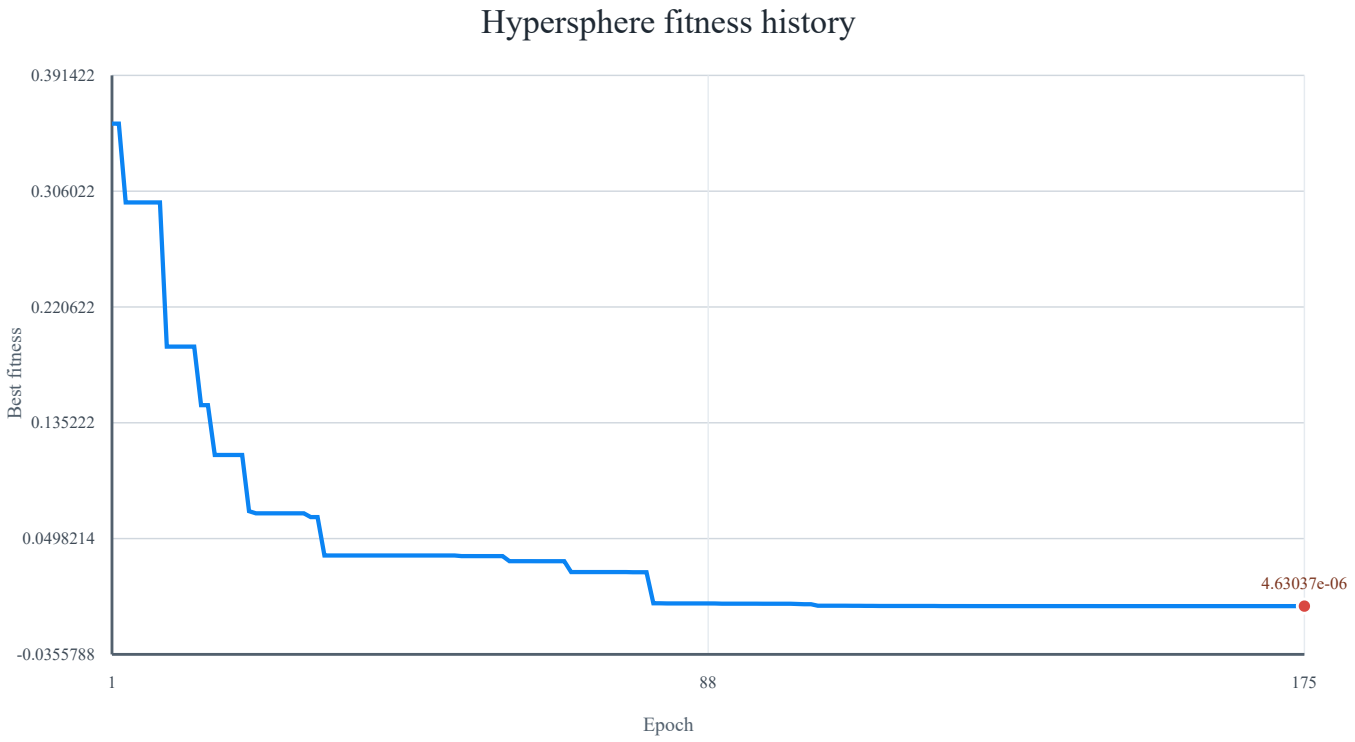
Wykres dla selekcji `roulette`

Kluczowa wartość configu względem pozostałych

- selection_method = roulette

Co widać na wykresie

- Również schodkowy przebieg, ale dłuższy
- Dojście bliskie rozwiązania dopiero koło 80 epoki



Wykres 4

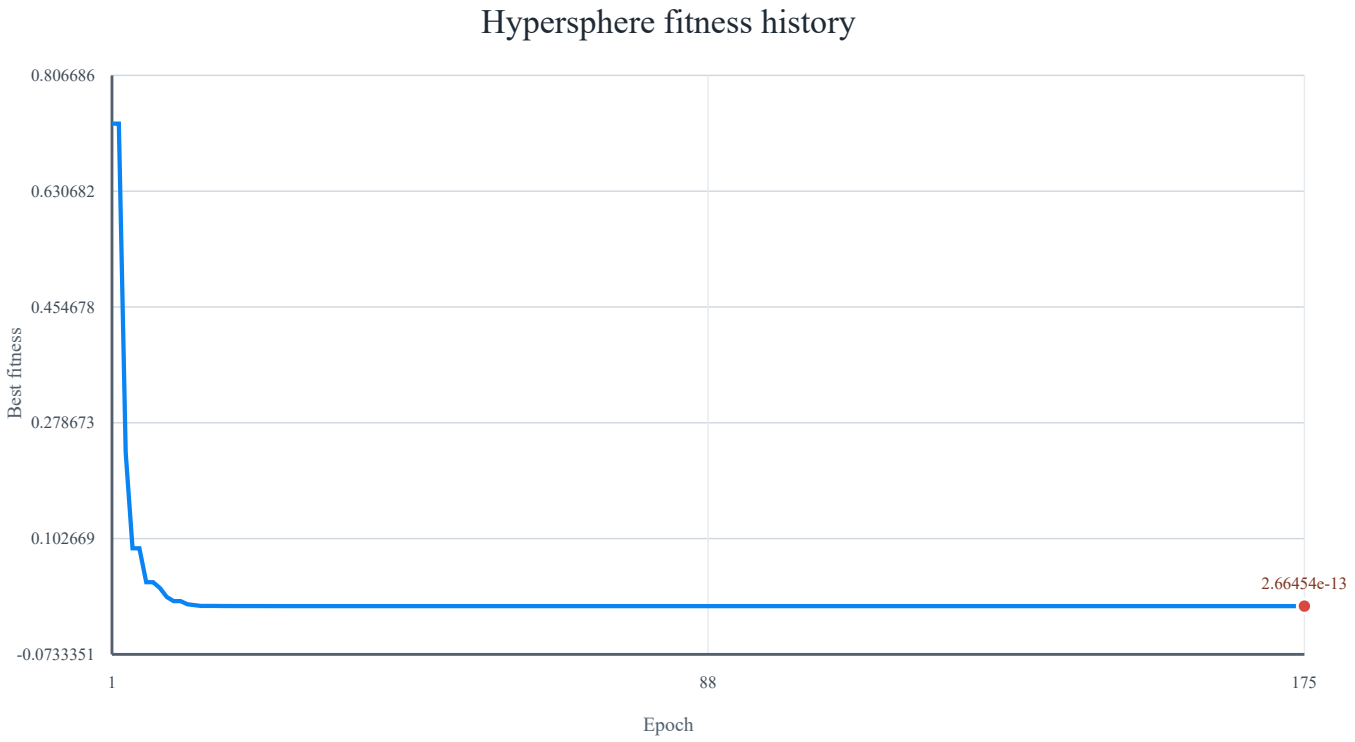
Wykres dla zwiększonej populacji `50 -> 500`

Kluczowa zmiana względem bazowego configu

- population_size: 50 -> 500

Co widać na wykresie

- Bardzo szybki spadek wartości fitness i dojście do bardzo małych wartości
- Praktyczna stabilizacja wyniku już około 10 epoki



Wykres 5

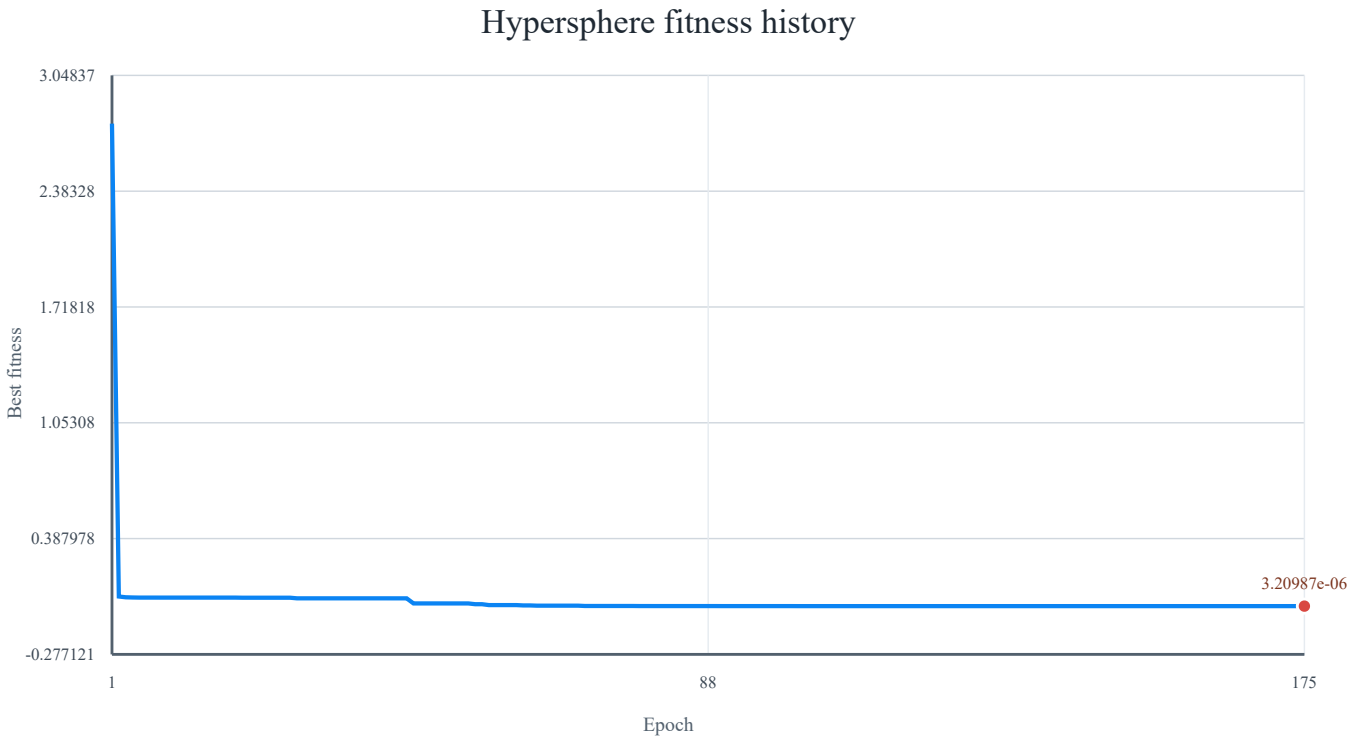
Wykres dla większego turnieju `3 -> 8`

Kluczowa zmiana względem bazowego configu

- tournament_size: 3 -> 8

Co widać na wykresie

- Wykres przypominający prostokąt, nagły spadek na samym początku, potem lekkie poprawki w kolejnych epokach



Wykres 6

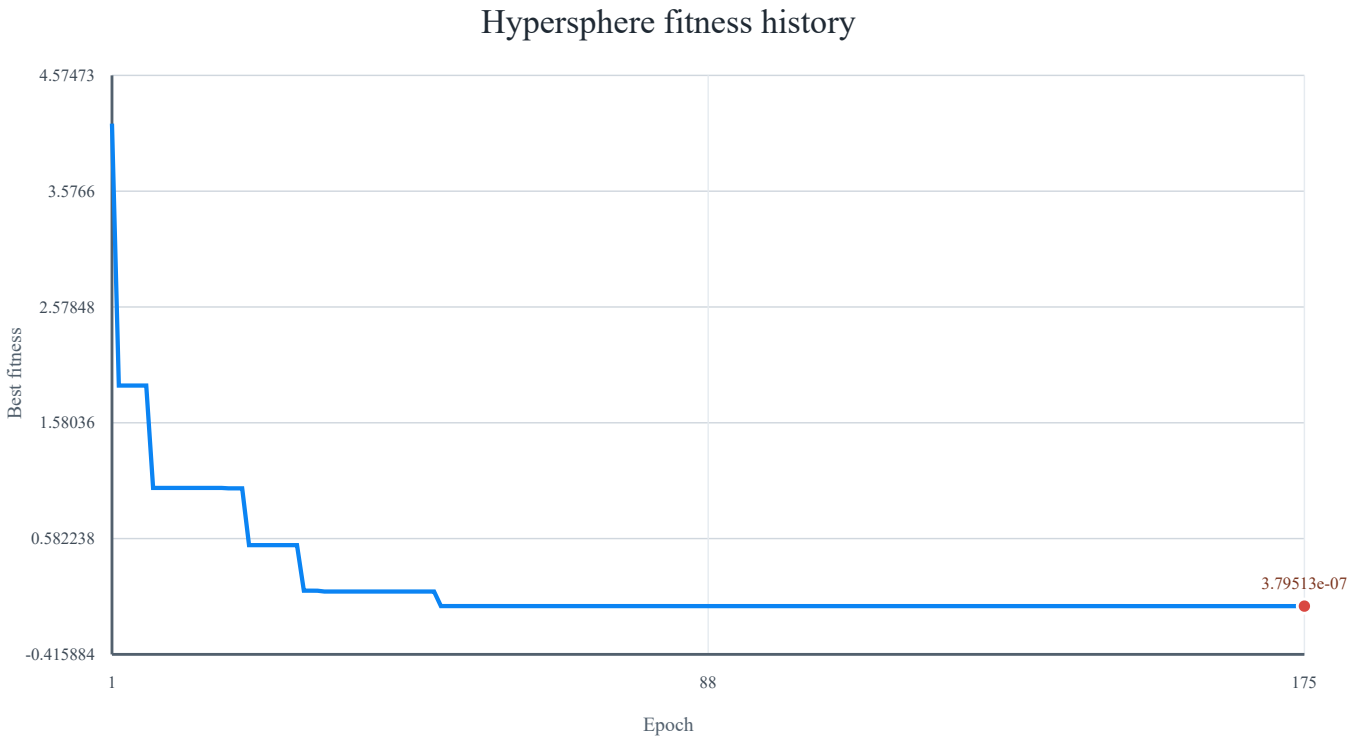
Wykres dla `granular crossover`

Kluczowa zmiana względem bazowego configu

- crossover_method: single_point -> granular
- grain_size: 5

Co widać na wykresie

- Bardzo długie fazy stagnacji przeplatane skokowymi poprawami



5. Porównanie wyników dla różnych konfiguracji algorytmu

Kierunek porównania konfiguracji

- Większa populacja zwykle poprawia eksplorację kosztem czasu obliczeń
- Większa liczba epok zwykle poprawia końcowy wynik kosztem czasu
- Selekcja tournament może dawać stabilne postępy przy poprawnym doborze rozmiaru turnieju. W naszym eksperymencie zbiegała ona najszybciej do poprawnego rozwiązania, natomiast miała potem problem ze znalezniem dokładniejszego rozwiązania.
- Zbyt wysoka mutacja pogarsza stabilizację rozwiązania
- Elityzm przyspiesza utrzymanie dobrych osobników ale może ograniczać różnorodność

Zestawienie końcowych wyników

Przebieg	selection_method	best_fitness	elapsed_seconds
Tournament	tournament	2.60e-05	0.12

Przebieg	selection_method	best_fitness	elapsed_seconds
Best	best	4.90e-06	0.10
Roulette	roulette	4.63e-06	0.11

Wnioski z porównania

- Sama zmiana metody selekcji istotnie zmieniła dynamikę zbieżności
- Selekcja `best` i roulette dały wyraźnie lepsze wartości końcowe niż tournament
- Tournament miał najszybszy progress na początku, jednak potem miał problem nadgonić
- `roulette` z najlepszym wynikiem zbiegało najwolniej, mimo to osiągnęło najlepszy wynik końcowy (ale niewiele lepszy niż `best`)

Dodatkowe porównanie: population, tournament_size i granular

Przebieg	Kluczowa zmiana	best_fitness	elapsed_seconds
Population 500	population_size 50 -> 500	2.66e-13	1.56
Tournament 8	tournament_size 3 -> 8	3.21e-06	0.15
Granular	crossover_method single_point -> granular (grain_size 5)	3.80e-07	0.16

Wnioski z dodatkowych 3 konfiguracji

- Zwiększenie populacji z 50 do 500 dało najszybszą i najbardziej stabilną poprawę jakości (gwałtowny spadek oraz szybka stabilizacja), a końcowo najlepszy wynik
- Dla tournament_size = 8 widać mocny spadek na początku, a potem głównie drobne poprawki; końcowy wynik był słabszy niż w konfiguracji granular i dużo słabszy niż dla population 500
- Crossover granular (grain_size = 5) miał długie fazy stagnacji przeplatane skokami poprawy; końcowo był lepszy od tournament_size = 8, ale gorszy od population 500
- W tej serii największy wpływ na jakość końcową miał rozmiar populacji, natomiast zmiana operatora krzyżowania i wielkości turnieju głównie zmieniała dynamikę zbieżności

6. Podsumowanie i analiza błędów

Podsumowanie

- Aplikacja GUI do optymalizacji funkcji Hypersphere algorytmem genetycznym
- Formularz do konfiguracji funkcji, zapis do plików, oraz generowanie wykresu
- Najlepsza dokładność wyniku ze wszystkich zebranych próbek uzyskana dla przebiegu `Population 500` , gdzie best_fitness = `2.66e-13`, za to kosztem największego czasu `1.5`s

Analiza błędów i ryzyk

- Jakość wyniku zależy od doboru parametrów i losowości procesu ewolucyjnego
- Praca z liczbami zmiennoprzecinkowymi (overflow itp.)
- Nie implementowaliśmy unit testów